

	Pràctica Entregable 01 Classe per encriptar informació	IMO30FP01 REV 00 11/13
---	---	---------------------------------------

CURS:	DAM2	DATA:	Setembre-2026	Qualificació	
MÒDUL:	M0490 - Programació Serveis i Processos				
TEMA:	Enciptació i desencriptació de dades				
RA	RA5				
Professors	Cristina Reina - Carles Bonet				

Enunciat:

Formeu part de l'equip de desenvolupament d'una empresa que necessita **enviar petits missatges protegits entre diferents aplicacions**.

L'empresa no vol que el programa principal conegui els detalls de com es protegeixen els missatges. Per aquest motiu, **us demana crear una classe criptogràfica independent i reutilitzable**.

La classe haurà de permetre:

MISSATGE + CLAU → Procés d'ENCRIPATACIÓ → MISSATGE ENCRIPATAT

MISSATGE ENCRIPATAT + CLAU → Procés de DESENCRIPATACIÓ → MISSATGE ORIGINAL

La vostra missió és **dissenyar el vostre propi sistema de xifrat** i implementar-lo com una classe reutilitzable.

Un exemple per entendre com crear el vostre propi sistema:

Suposem que tenim:

- Missatge: HOLA
- Clau: 3

Podem utilitzar la clau per desplaçar cada lletra tres posicions:

- $H \rightarrow K$
- $O \rightarrow R$
- $L \rightarrow O$
- $A \rightarrow D$

Per tant: HOLA → KROD

Per recuperar el missatge original, fem el procés invers: KROD → HOLA.

1. Investiga el funcionament

Respon aquestes preguntes:

- Què passa si encriptem dues vegades el mateix missatge amb la mateixa clau?

Depenent del sistema de encriptació, si és de desplaçament doncs la clau acabarà sent el doble de la clau original.

- Què passa si canviem la clau? Prova HOLA + clau 3 i HOLA + clau 7.

Doncs que tindrem dos missatges xifrats diferents encara que tinguin una clau diferent.

HOLA + clau 3 -> **KROD**

HOLA + clau 7 -> **OVSH**

- Què passa si intentem descriptar utilitzant una clau incorrecta?

Tindrem un missatge ilegible.

Com a mínim haurà de complir:

- El mateix missatge amb la mateixa clau ha de produir el mateix resultat.
- Una clau diferent ha de modificar el resultat.
- Si utilitzem la clau correcta, hem de poder recuperar exactament el missatge original.

$\text{Descripta}(\text{Encripta}(\text{"HOLA"}, \text{"123"}), \text{"123"}) = \text{"HOLA"}$

2. Dissenya l'algoritme propi d'encriptació i descriptació

Abans de programar, heu de dissenyar el vostre sistema de xifrat.

Podeu utilitzar, per exemple:

- un sistema de substitució.
- un sistema de desplaçament.
- una combinació de diferents operacions.

La clau ha de participar obligatòriament en el procés.

Nom del vostre sistema	Xifrat P+K
Tipus de sistema	☐ Substitució ☐ Desplaçament ☒ Combinació ☐ Altres: _____
Tipus de clau	Númerica
Com funciona l'encriptació?	

	El xifrat comença intercanviant de dos en dos l'ordre de les lletres, després depenent de si la posició de la lletra és parell o imparell se li sumarà o se li restarà la clau en l'ordre alfabètic. Ex (clau 4): H -> O = O+4 -> K O -> L = L+4 -> P L -> W = W+4 -> A A -> E = E+4 -> I
Com funciona la descriptació?	La descriptació funciona igual però al invers, sapiguent la clau es fa la mateixa operació: K -> H = K-4 = O -> L
Com sé que el meu sistema funciona?	Doncs perquè si encriptem un missatge amb una clau i al fer el recorregut invers només sapiguent la clau i el missatge encriptat ens dona el missatge descriptat, a més si utilitzem un altre clau que no és correcte ens donara un missatge ilegible.

3. Demuestra el teu algoritme

Abans de començar a programar, feu un exemple manual complet.

Missatge: HOLA

Clau: _____4_____

Encriptació: H → _K_ O → _L_ L → _W_ A → _P_

Resultat: _____KLWP_____

Igualment, descripta el resultat i demostra que recuperes HOLA.

4. Implementació de la classe "Encripta" en Java

Crea una classe independent anomenada:

ClasseCriptografica

La classe haurà d'oferir, com a mínim, aquests dos mètodes:

- `Encripta(Missatge, Clau)`
- `Desencripta(Missatge, Clau)`

Els mètodes han de retornar el missatge resultant.

La classe criptogràfica no ha de contenir el programa principal. El programa principal serà qui utilitzarà la classe.

Programa principal → ClasseCriptografica → Encripta / Desencripta

5. Proves del sistema dissenyat d'encriptació

Test	Prova	Resultat
Test 1	Missatge curt: HOLA	☒ Correcte ☐ Incorrecte Observacions: _____
Test 2	Missatge amb espais: HOLA MÓN	☒ Correcte ☐ Incorrecte Observacions: _____
Test 3	Missatge més llarg: almenys 30 caràcters	☒ Correcte ☐ Incorrecte Observacions: _____
Test 4	Clau diferent: comprovar que el resultat canvia	☒ Correcte ☐ Incorrecte Observacions: _____
Test 5	Encriptar i desencriptar: recuperar exactament el missatge original	☒ Correcte ☐ Incorrecte Observacions: _____
Test 6	Clau incorrecta: explicar què passa	☒ Correcte ☐ Incorrecte Observacions: Em dona un missatge diferent que el original ja que es mou un nombre diferent de lletres.

6. Implementació d'un sistema de xifrat AES en Java

Fins ara heu dissenyat i implementat el vostre propi sistema de xifrat. Ara fareu un pas més: **utilitzareu AES (Advanced Encryption Standard)**, un algoritme de xifrat simètric utilitzat en sistemes reals.

En aquesta part **no haureu de crear l'algoritme AES des de zero**. Java proporciona una API de criptografia que permet utilitzar-lo.

Objectiu

Crear una classe anomenada **ClasseAES** que permeti:

MISSATGE + CLAU

↓

AES

↓

MISSATGE XIFRAT

i recuperar posteriorment el missatge original:

MISSATGE XIFRAT + CLAU

↓

AES

↓

MISSATGE ORIGINAL

La classe haurà d'oferir, com a mínim, els dos mètodes següents:

```
public static String encripta(String missatge, String clau)
```

```
public static String desencripta(String missatgeXifrat, String clau)
```

6.1. Eines disponibles

Per realitzar aquesta activitat podeu utilitzar les classes i mètodes següents de Java:

Classes:

```
import javax.crypto.Cipher;
```

```
import javax.crypto.spec.SecretKeySpec;
```

```
import java.nio.charset.StandardCharsets;
```

```
import java.util.Base64;
```

Mètodes i elements que podeu consultar i utilitzar:

```
getBytes()
```

```
Cipher.getInstance()  
  
cipher.init()  
  
cipher.doFinal()  
  
Base64.getEncoder()  
  
Base64.getDecoder()  
  
SecretKeySpec  
  
Cipher.ENCRYPT_MODE  
  
Cipher.DECRYPT_MODE
```

Aquestes són les eines que necessitareu per construir el vostre sistema.

No es proporcionarà el codi complet de la classe. Haureu d'investigar el funcionament de les classes i mètodes proporcionats i decidir com combinar-los per construir els processos d'enciptació i desenciptació

Trobareu un [annex explicatiu](#) breu al final del document.

6.2. Implementa la classe **ClasseAES**

Crea una classe independent anomenada [ClasseAES](#), que tingui els mètodes:

[encripta \(missatge, clau\)](#)
[desencripta \(missatge, clau\)](#)

El mètode [encripta\(\)](#) haurà de:

1. Rebre un missatge i una clau.
2. Preparar la clau perquè pugui ser utilitzada per AES.
3. Crear i configurar el sistema de xifrat.
4. Convertir el missatge al format necessari.
5. Xifrar el missatge.
6. Convertir el resultat en un format que es pugui retornar com a **String**.
7. Retornar el missatge xifrat.

El mètode [desencripta\(\)](#) haurà de realitzar el procés invers:

1. Rebre el missatge xifrat i la clau.
2. Preparar la clau.
3. Crear i configurar el sistema de desxifrat.
4. Recuperar les dades xifrades.

5. Desxifrar-les.
6. Convertir el resultat novament a text.
7. Retornar el missatge original.

6.3. Programa principal

Crea un `ProgramaPrincipal` que utilitzi la classe `ClasseAES`.

Utilitza inicialment les dades següents:

`Missatge: Aquest és un missatge secret.`

`Clau: 1234567890123456`

El programa haurà de:

1. Mostrar el missatge original.
2. Encriptar-lo utilitzant AES.
3. Mostrar el missatge xifrat.
4. Desencriptar el resultat utilitzant la mateixa clau.
5. Mostrar el missatge recuperat.

El resultat final ha de demostrar que:

```
Desencripta(Encripta(missatge, clau), clau) = missatge
```

6.4. Experimenta amb el teu programa

Una vegada el programa funcioni, realitza les proves següents:

Prova 1. Clau correcta

Comprova que el missatge original es recupera correctament.

Prova 2. Clau diferent

Intenta desencriptar el missatge utilitzant una clau diferent.

Explica què passa.

Dona un error intern i no retorna cap missatge perquè diu que el bloc final no coincideix amb l'estàndard de AES.

Prova 3. Missatge diferent

Utilitza un altre missatge i comprova que el procés continua funcionant.

Prova 4. Clau de longitud incorrecta

Prova d'utilitzar una clau que no tingui la longitud adequada.
Observa què succeeix i explica el motiu.

Dona error dient que la clau no té la mida corresponent per una clau AES i no retorna cap missatge.

6.5. Comprèn el codi

No n'hi ha prou amb aconseguir que el programa funcioni. Has de poder explicar què està fent.

Respon les preguntes següents:

1. Quina classe de Java s'encarrega de realitzar el xifrat?

Cipher

2. Per a què serveix `SecretKeySpec`?

Per crear la clau del sistema AES.

3. Què fa `Cipher.getInstance()`?

Crea i selecciona el mètode amb l'algoritme d'enciptació.

4. Quina diferència hi ha entre `ENCRYPT_MODE` i `DECRYPT_MODE`?

ENCRYPT_MODE: Serveix per encriptar un missatge.

DECRYPT_MODE: Serveix per desencryptar un missatge xifrat.

5. Per a què serveix `doFinal()`?

Executa el mètode de xifrat.

6. Per què hem de convertir el missatge a bytes?

Perquè els algoritmes de encriptació no saben utilitzar ni reconèixer lletres, a més així s'aconsegueix una estandardització en el format amb el que arriben els missatges per xifrar, ja que no tenim que anar amb compte de si és UTF-8, UTF-16 etc...

7. Per què utilitzem Base64?

Per la integritat de les dades, ja que Base64 fa servir només caràcters compatibles amb tots els sistemes, a més com és només una cadena de text és més fàcil de transportar.

8. Quina mida té la clau que has utilitzat?

16 caràcters que són 128 bits.

9. Què significa que AES sigui un sistema de xifrat simètric?

Que només hi ha una clau i és xifrabla i desxifrabla bidireccionalment.

10. Per què necessitem la mateixa clau per recuperar el missatge?

Perquè a diferència dels xifrats asimètrics, els simètrics utilitzen la mateixa clau per xifrar i desxifrar.

Finalment, respon:

Per què és preferible utilitzar un algoritme criptogràfic estàndard com AES en lloc de crear el nostre propi sistema de xifrat?

Perquè els algoritmes ja estan creats i testejats des de anys, si creem un sistema nosaltres de 0 no tindrà el mateix nivell de seguretat i rendiment.

Lliurament

Heu de lliurar en un repositori GitHub:

1. Sistema de xifrat propi

- `ClasseCriptografica.java`
- `ProgramaPrincipal.java`

2. Implementació AES

- `ClasseAES.java`
- `ProgramaPrincipalAES.java`

3. Documentació

- Document PDF amb les respostes i explicacions de la pràctica.

Cal lliurar aquesta documentació en un Repositori GitHub i que el compartiu amb la Cristina i en Carles.

Xifrat AES

El xifrat AES (Advanced Encryption Standard) va ser aprovat com estàndard de xifrat pel govern dels EUA després d'un procés de 5 anys. El xifrat va ser desenvolupat per dos criptòlegs belgues J.Daemen i V. Rijmen que el van enviar amb el nom de Rijndael.

Característiques:

- Xifrat de bloc: mida 128 bits.
- Clau de xifrat de 128,192 o 256 bits.

Actualment es considera un sistema de xifrat segur (la NSA exigeix clau de 192 o 256 bits per documents d'alta seguretat). És un algoritme ràpid, AES 256 és més ràpid que DES i per tant molt més ràpid que la solució 3DES.

Elements del xifrat:

- IV (Vector inicialització): És necessari per iniciar cada procés de xifrat. Té una mida de 16 bytes (1 bloc). El seu objectiu és evitar que dos blocs iguals tinguin el mateix criptograma.
- Clau (Key): És la principal informació per xifrar i desxifrar en els algoritmes simètrics. A la implementació per defecte de .NET s'utilitza la de 256 bytes.

Cipher mode:

ECB: xifra cada bloc per separat. Per tant, blocs idèntics donen el mateix criptograma.

CBC: s'utilitza un vector d'inicialització (IV) i cada bloc es combina amb el bloc anterior amb una operació XOR. Això assegura que blocs iguals no donin un criptograma idèntic.

Clau de xifrat:

Existeixen mètodes per obtenir la clau de forma automàtica, però això només té sentit si aquesta clau es pot enviar de forma segura. Volem obtenir la clau a partir de passphrase que introdueixi l'usuari.

La forma més còmoda per l'usuari és introduir una password o passphrase enlloc de una clau i un vector d'inicialització. Utilitzem `Rfc2898DeriveBytes` per obtenir un *hash* de la combinació dels dos textos. S'afegeix una cadena *salt* per fer més difícil l'atac per RainBow Tables. L'algoritme es pot aplicar iterativament per augmentar la complexitat.

Rúbrica:

Criteri	Què s'avalua	Punts	
1. Disseny del sistema propi	Defineix l'algoritme, la clau i explica correctament encriptació i desencriptació	1,5	
2. Implementació del sistema propi	ClasseCriptografica funciona correctament i està separada del programa principal	1,5	
3. Ús de la clau	La clau participa realment en el procés i una clau incorrecta no permet recuperar correctament el missatge	1,0	
4. Proves del sistema propi	Realitza i documenta correctament els tests proposats	1,0	
5. Implementació d'AES	Utilitza correctament l'API de Java i implementa ClasseAES amb encriptació i desencriptació	2,0	
6. Comprensió de l'API criptogràfica	Explica Cipher, SecretKeySpec, init(), doFinal(), Base64, etc.	1,0	
7. Experimentació amb AES	Realitza les proves amb diferents claus i missatges i explica els resultats	0,75	
8. Documentació i comparació	Respon les preguntes i compara el sistema propi amb AES de manera raonada	0,75	
9. Organització i lliurament	Estructura correcta del projecte, noms de classes, repositori GitHub i documentació	0,5	
TOTAL		10	